

Security: Docker Security

We will look at the various concepts related to security in Docker. Let us start with a **host with Docker installed on it**. This host has a set of its own processes running such as a number of operating system processes, the docker daemon itself, the SSH server etc. We will now run an **Ubuntu docker container that runs a process that sleeps for an hour**.

We know that unlike virtual machines **containers are not completely isolated from their host. Containers and the hosts share the same kernel**. Containers are isolated using namespaces in Linux. The host has a namespace and the containers have their own namespace. All the processes run by the containers are in fact run on the host itself, but in their own namespaces.

As far as the **docker container is concerned**, it is in its own namespace and it can see its own processes only, cannot see anything outside of it or in any other namespace. So, when you list the processes from within the docker container you see the sleep process with a process ID of 1.

For the **docker host**, all processes of its own as well as those in the child namespaces are visible as just another process in the system. So, when you list the processes on the host you see a list of processes including the sleep command, but with a different process ID. This is because the **processes can have different process IDs in different namespaces and that's how Docker isolates containers within a system. So that's process isolation**.

Security - Users



```
ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
project   3720  0.1  0.1  95500 4916 ?        R    06:06   0:00 sshd: project@pts/0
project   3725  0.0  0.1  95196 4132 ?        S    06:06   0:00 sshd: project@notty
project   3727  0.2  0.1  21352 5340 pts/0    Ss   06:06   0:00 -bash
root      3802  0.0  0.0   8924 3616 ?        Sl   06:06   0:00 docker-containerd-
shim
root      3816  1.0  0.0   4528  828 ?        Ss   06:06   0:00 sleep 3600
```

```
ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0   4528  828 ?        Ss   03:06   0:00 sleep 3600
```

```
docker run --user=1001 ubuntu sleep 3600
```

```
ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
1001         1  0.0  0.0   4528  828 ?        Ss   03:06   0:00 sleep 3600
```

ud.com

Let us now look at users in context of security. The docker host has a set of users, **a root user as well as a number of non-root users. By default, docker runs processes within containers as the root user (interact with the kernel).** This can be seen in the output of the commands we ran earlier. Both within the container and outside the container on the host, the process is run as the root user. Now if you do not want the process within the container to run as the root user, you may set the user using the user option with the docker run command and specify the new user ID. You will see that the process now runs with the new user id. Let us take a step back. What happens **when you run containers as the root user? Is the root user within the container the same as the root user on the host?** Can the process inside the container do anything that the root user can do on the system? If so, isn't that dangerous? Well, **docker implements a set of security features that limits the abilities of the root user within the container. So, the root user within the container isn't really like the root user on the host.**

Security Context

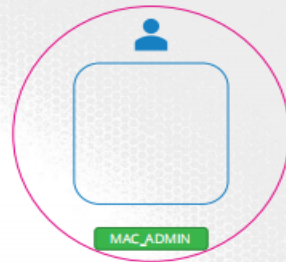
As you know already, in Kubernetes containers are encapsulated in PODs. You may choose to configure the **security settings at a container level or at a POD level.** If you configure it at a POD level, the settings will carry over to all the containers within the POD. If you configure it at both the POD and the Container, **the settings on the container will override the settings on the POD.**

Let us start with a POD definition file. This pod runs an ubuntu image with the sleep command. To configure security context on the container, add a field called securityContext under the spec section of the pod. Use the **runAsUser** option to set the user ID for the POD.



Security Context

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  securityContext:
    runAsUser: 1000
  containers:
    - name: ubuntu
      image: ubuntu
      command: ["sleep", "3600"]
```



KodeKloud.com

To set the same configuration on the container level, move the whole section under the container specification. To add capabilities, use the capabilities option and specify a list of capabilities to add to the POD.

Security Context

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  containers:
    - name: ubuntu
      image: ubuntu
      command: ["sleep", "3600"]
      securityContext:
        runAsUser: 1000
        capabilities:
          add: ["MAC_ADMIN"]
```



KodeKloud.com

Note: Capabilities are only supported at container level and not to Pod level.

Security Primitive?

Image Security

| Image

```
nginx-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx
    image: nginx
```

| Image

`image: docker.io/nginx/nginx`



`gcr.io/kubernetes-e2e-test-images/dnsutils`

| Private Repository

```
docker login private-registry.io
```

Login with your Docker ID to push and pull images from Docker Hub. If you don't have a Docker ID, head over to <https://hub.docker.com> to create one.

Username: registry-user

Password:

WARNING! Your password will be stored unencrypted in `/home/vagrant/.docker/config.json`.

Login Succeeded

```
docker run private-registry.io/apps/internal-app
```

How to pass the private repository credentials to kubernetes worker node?

step1: create a secret Object (regcred) with the following data

```
-- docker-server=private-registry.io  
-- docker-username=registry-user  
--docker-password=registry-password  
--docker-email=registry-user@org.com
```

Step2: add the spec. **imagePullSecrets**

Private Repository

```
docker login private-registry.io
```

```
docker run private-registry.io/apps/internal-app
```

```
kubectl create secret docker-registry regcred \
```

```
--docker-server= private-registry.io \
```

```
--docker-username= registry-user \
```

```
--docker-password=registry-password \
```

```
--docker-email= registry-user@org.com
```

```
nginx-pod.yaml
```

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: nginx-pod  
spec:  
  containers:  
  - name: nginx  
    image: private-registry.io/apps/internal-app  
  imagePullSecrets:  
  - name: regcred
```

Security in K8s: secure host

When we think about security, there are two concepts

- Authentication: Who are you
- Authorization: What you are allowed to do



- ☐ Password based authentication disabled
- ☐ SSH Key based authentication

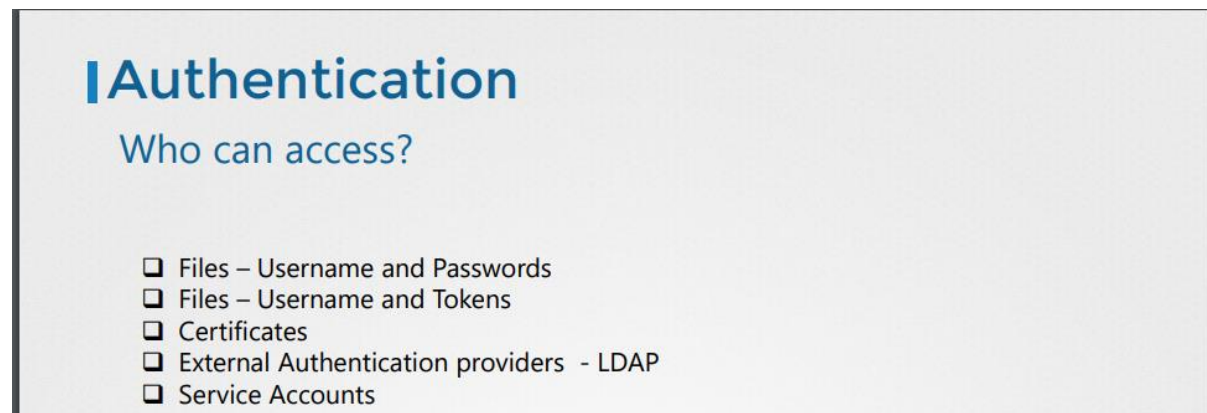
By default, k8s implements SSL/TLS. When two parties are communicating in secure way.

In K8s to perform any activity we use kubectl client utility, interact with the api-server, first authentication starts from here, first api-server authenticates the kubectl utility.

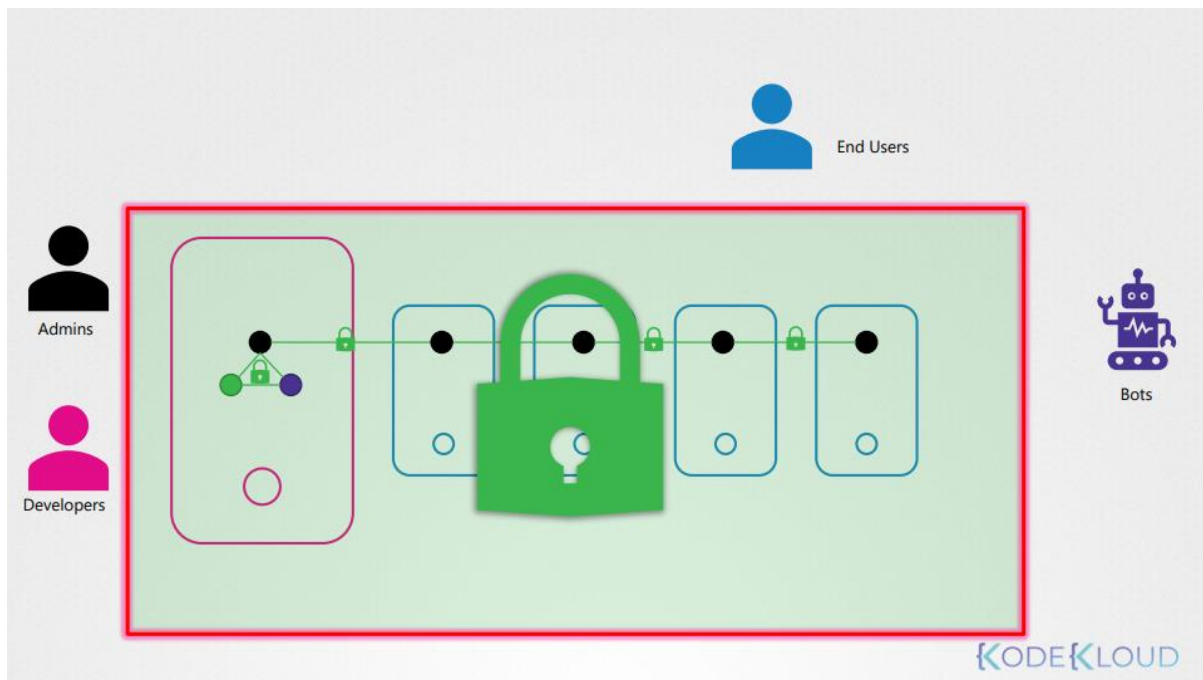
step1: Controlling access to api-server itself.



Authentication



The K8s cluster consists of multiple nodes physical or virtual and various components that work together



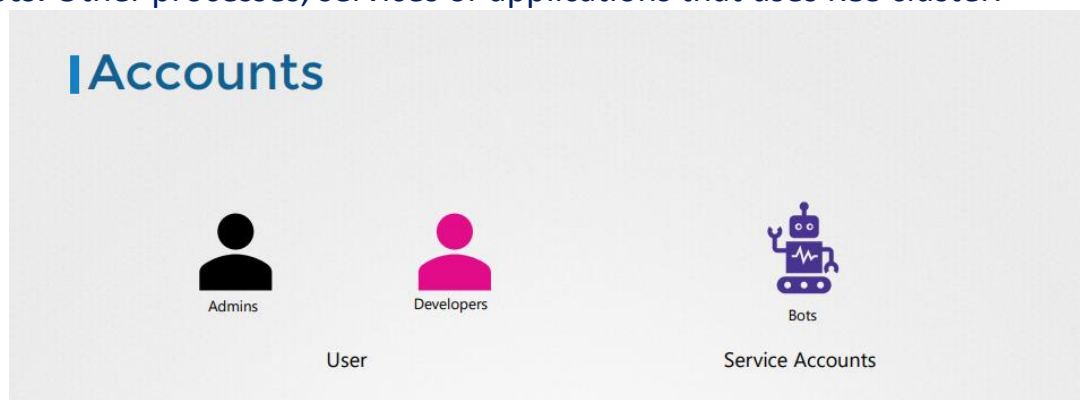
Security to access to the k8s cluster, by end users who access the cluster are managed by the application itself internally.

Our objective of Users who access the cluster for administrative purpose.

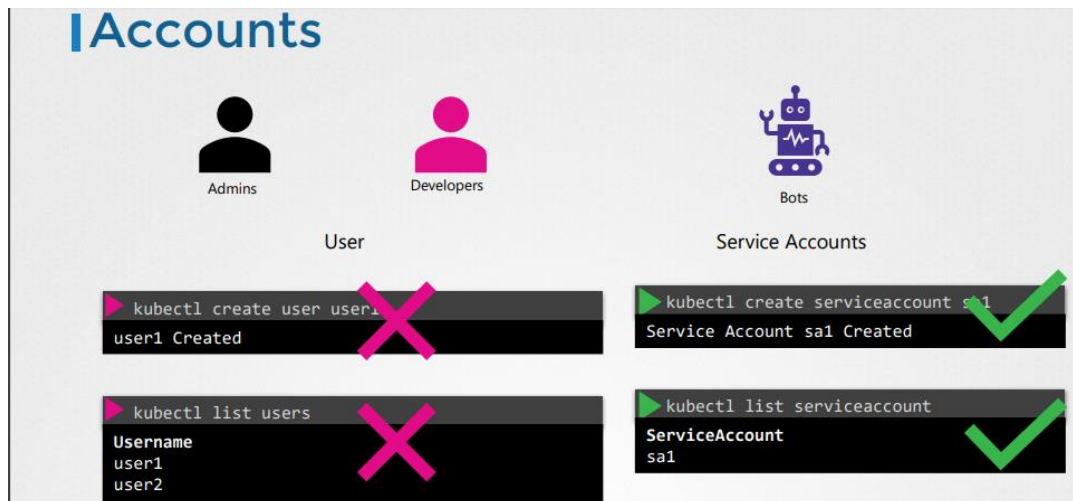
We have two types of users

Human: Such as administrator and developer

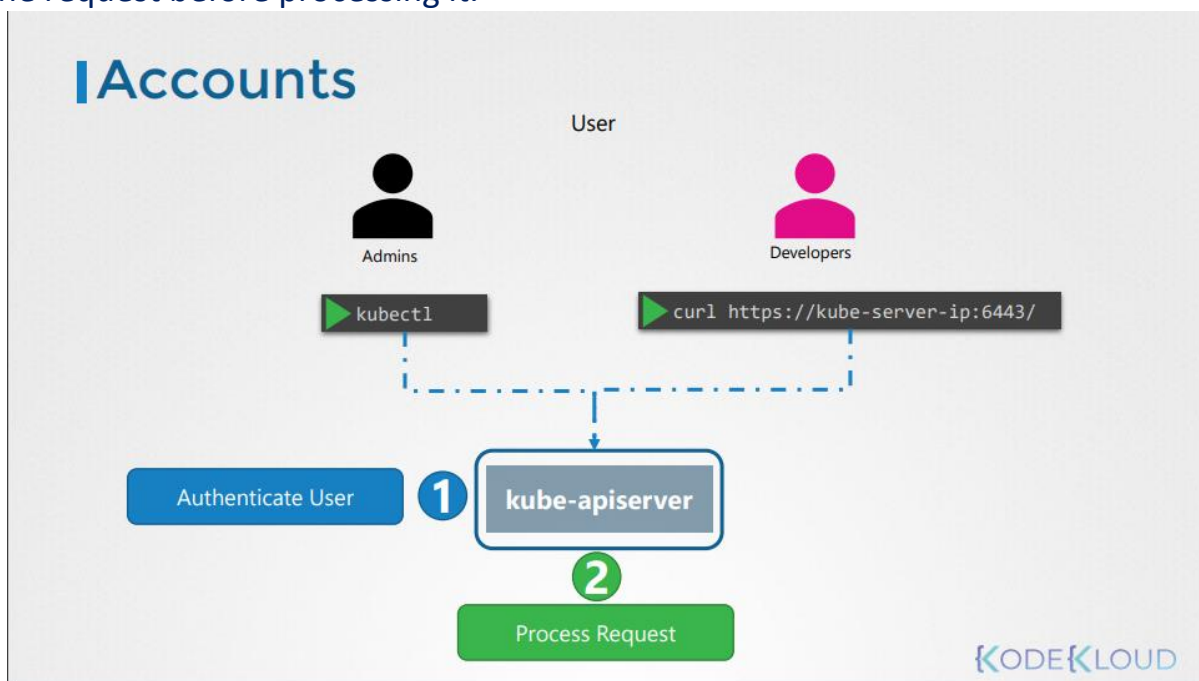
Robots: Other processes, services or applications that uses K8S cluster.



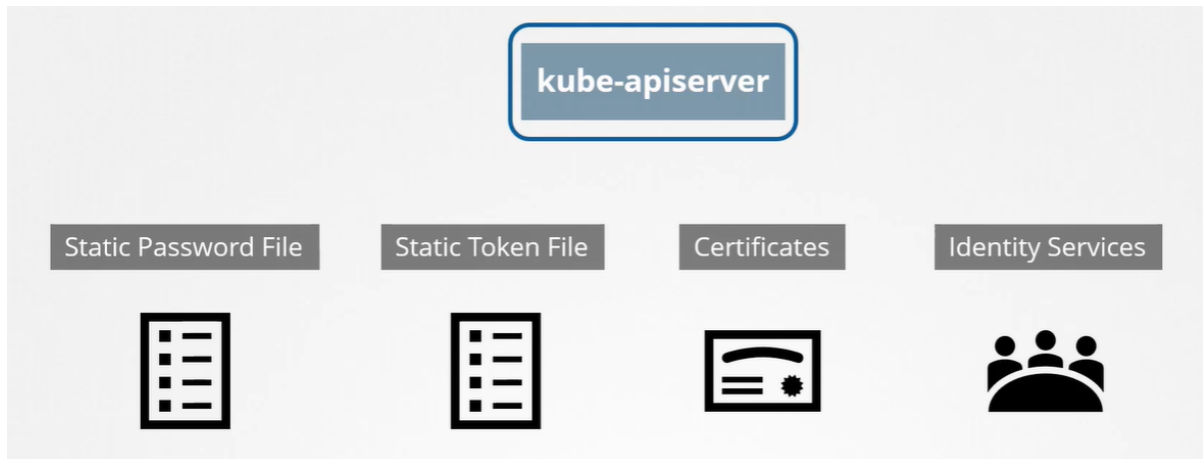
K8s doesn't manage the User accounts natively, it relies on external resources like file that includes user details, or certificates or third-party Identity service like LPAD to manage these users. However, in case of service account K8s can manage them. Can manage service account using K8s API.



All user access is managed by the kube-api-server, kube-apiserver authenticates the request before processing it.



All request comes through the api-server, how kube-api-server authenticates?



Different ways of Authentications

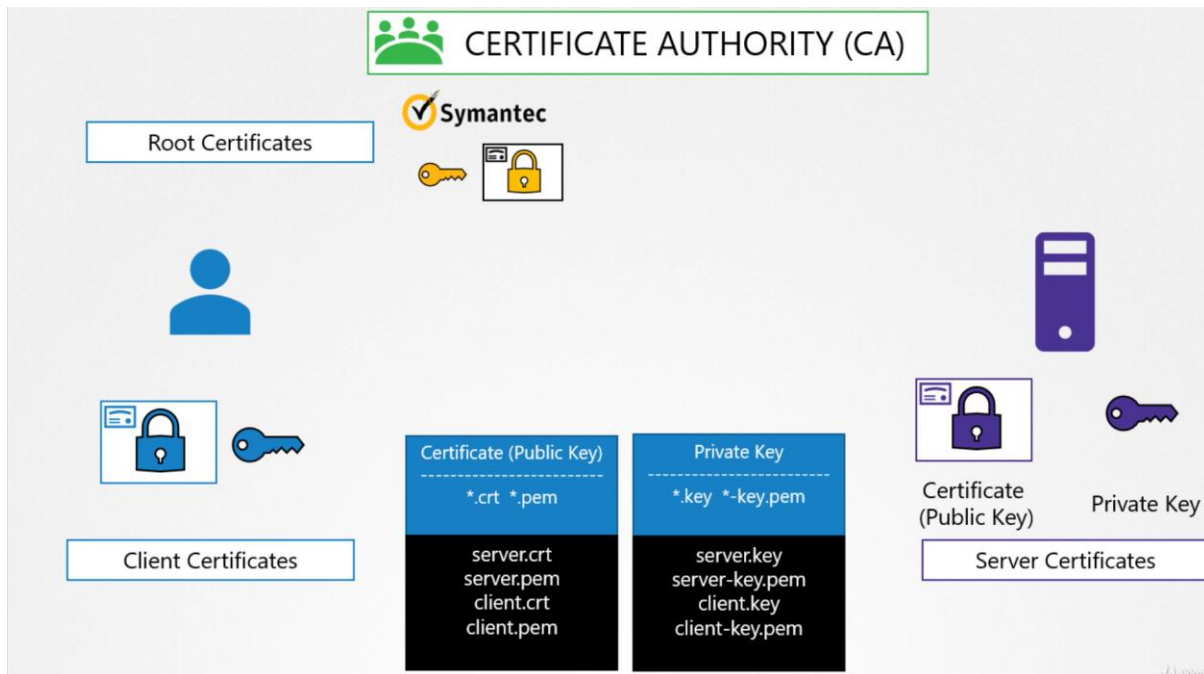
- User, Password
- User, Tokens
- **Certificates**
- **Service Accounts**

kubeadm tool automatically restart the kube-apiserver once you update this file. Keeping userid and password in config file is not a good approach. We have to go for certificate.

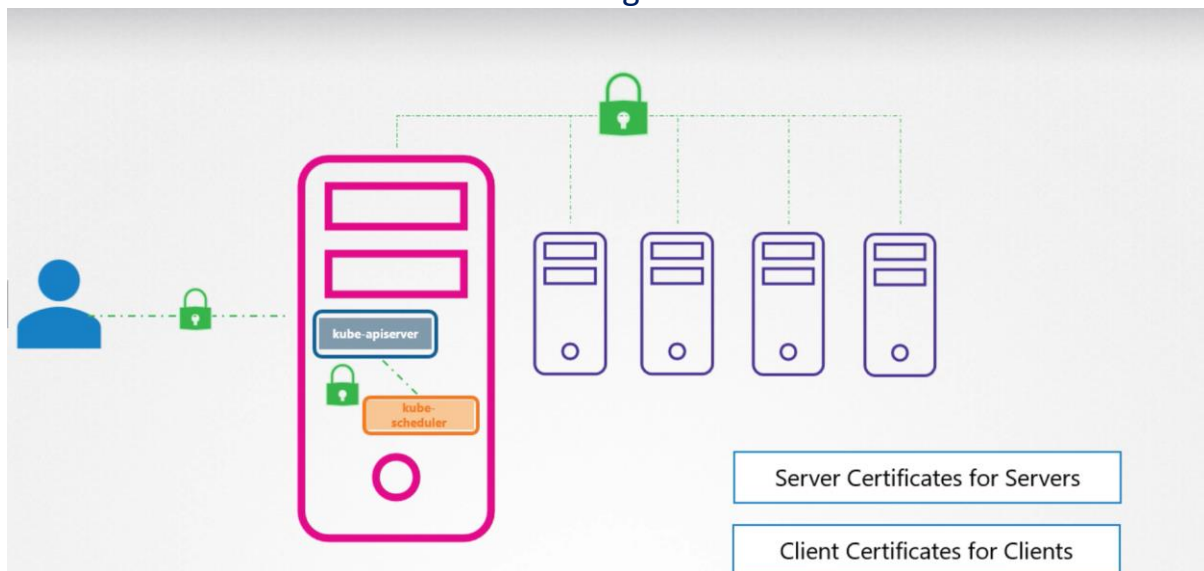
TLS certificates in kubernetes.

There are three types of certificates

- **Server certificate:** configured on server
- **Root certificates:** configured on CA servers
- **Client certificate:** configured on the client



K8s cluster consists of a set of master and worker nodes, communication among them must be secure and encrypted. All interactions between all services and their clients must need to be secure through SSL.



For example, administrator interacting with api-server through kubectl utility must be establish a TLS secure connection, also communication between all the components with in K8s also need to be secure.

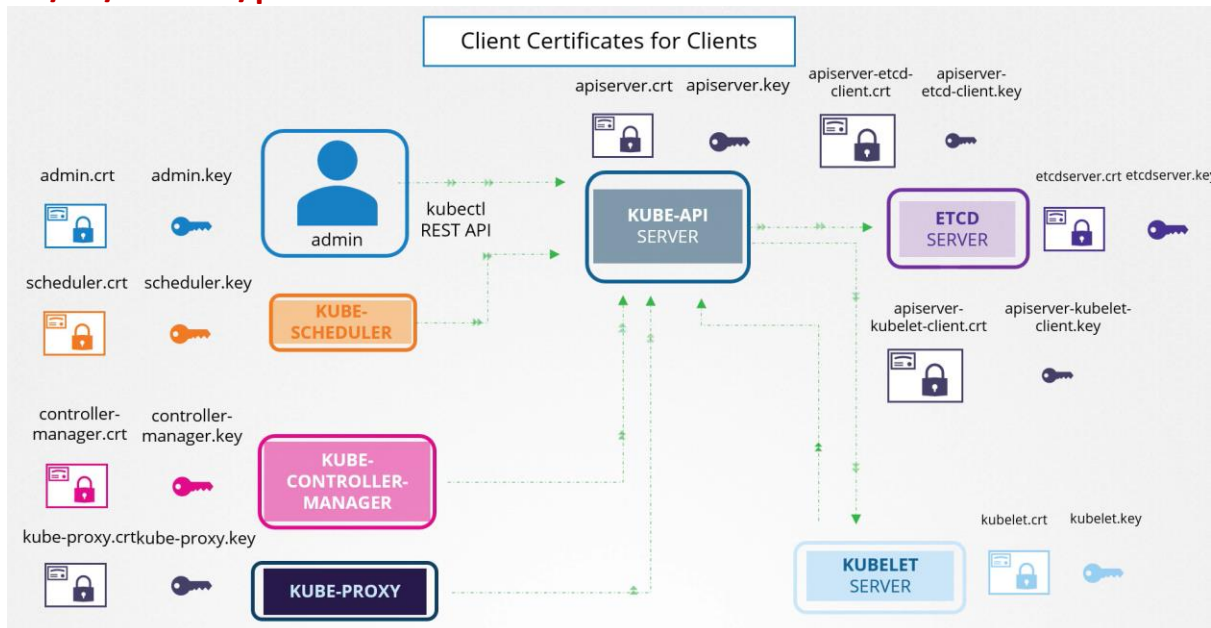
Server certificates for servers

On Master: apiserver, ETCD

/etc/kubernetes/manifests/pki

On Worker: kubelet

/var/lib/kubelet/pki



By default, K8s is its own CA.

To generate the certificate there are different tools are available



We use OPENSSL tool to generate the certificate.

TLS in K8s – Generate Certificates

step1: create a prod-user

`mkdir -p /home/prod-user/certs`

`cd /home/prod-user/certs`

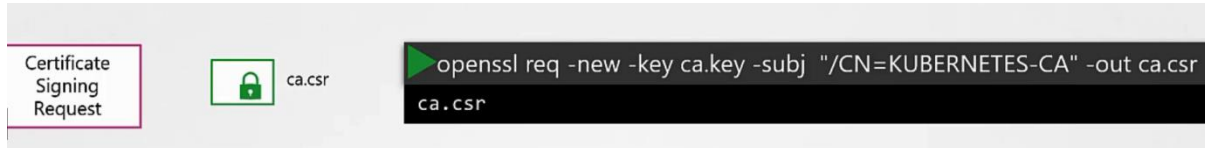
step2: create a private key using openssl command

`openssl genrsa -out prod-user.key 2048`



Step3: create a csr (certificate signing request) using the key generate in step2
.csr is just like a certificate with all of your details but with no sign

`openssl req -new -key prod-user.key -out prod-user.csr # common name,
organization name`



Step4: certificate signing request is the base64 encoded value of the CSR file content. You can get the content using this command:

`cat prod-user.csr | base64 | tr -d "\n"`

step4.1: create a csr manifests

`nano prod-user. yaml`

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: prod-user
spec:
  groups:
  - system:authenticated
  request: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSBRSRVFRVNVULS0tLS0KTU1JQ12qQ0NBVDROQVFBd0VURVBNQTBHQTFVRUF3d0d2VzVuWld4aE1JSUJJa0FOQmdrcWhraUc5dzBCQVFFRgpBQU
  signerName: kubernetes.io/kube-apiserver-client
  usages:
  - client auth
```

step4.2: create a csr object

`kf prod-user. yaml`

step4.3: validate the csr object

`kg csr # status pending approval`

step4.4: approve the certificate

`k certificate approve prod-user # csr approved`

We follow the same process to generate the client certificate for other components who access the K8s cluster.

View Certificate Details?

```
cat /etc/kubernetes/manifests/kube-apiserver.yaml

spec:
  containers:
  - command:
    - kube-apiserver
    - --authorization-mode=Node,RBAC
    - --advertise-address=172.17.0.32
    - --allow-privileged=true
    - --client-ca-file=/etc/kubernetes/pki/ca.crt
    - --disable-admission-plugins=PersistentVolumeLabel
    - --enable-admission-plugins=NodeRestriction
    - --enable-bootstrap-token-auth=true
    - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
    - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.crt
    - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key
    - --etcd-servers=https://127.0.0.1:2379
    - --insecure-port=0
    - --kubelet-client-certificate=/etc/kubernetes/pki/apiserver-kubelet-client.crt
    - --kubelet-client-key=/etc/kubernetes/pki/apiserver-kubelet-client.key
    - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
    - --proxy-client-cert-file=/etc/kubernetes/pki/front-proxy-client.crt
    - --proxy-client-key-file=/etc/kubernetes/pki/front-proxy-client.key
    - --secure-port=6443
    - --service-account-key-file=/etc/kubernetes/pki/sa.pub
    - --service-cluster-ip-range=10.96.0.0/12
    - --tls-cert-file=/etc/kubernetes/pki/apiserver.crt
    - --tls-private-key-file=/etc/kubernetes/pki/apiserver.key
```

View Certificate Validity Details?

/etc/kubernetes/pki/apiserver.crt

```
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -text -noout

Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number: 3147495682089747350 (0x2bae26a58f090396)
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: CN=kubernetes
    Validity
      Not Before: Feb 11 05:39:19 2019 GMT
      Not After : Feb 11 05:39:20 2020 GMT
    Subject: CN=kube-apiserver
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:d9:69:38:80:68:3b:b7:2e:9e:25:00:e8:fd:01:

        Exponent: 65537 (0x10001)
    X509v3 extensions:
      X509v3 Key Usage: critical
        Digital Signature, Key Encipherment
      X509v3 Extended Key Usage:
        TLS Web Server Authentication
      X509v3 Subject Alternative Name:
        DNS:master, DNS:kubernetes, DNS:kubernetes.default,
        DNS:kubernetes.default.svc, DNS:kubernetes.default.svc.cluster.local, IP
        Address:10.96.0.1, IP Address:172.17.0.27
```

What happens when certificate expires?

Every time Certificate Expire, we have to generate a new csr, and is getting signed by CA.

What is a CA server?

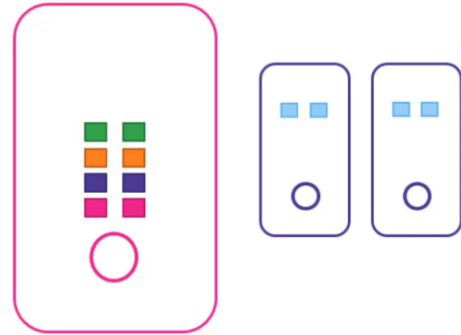
CA server is just pair of keys and certificate files, whoever access to this pair of files can sign the certificate.

These files need to be protected and keep in safe environment?



As of now certificates are stored in master node hence master node is also our CA server.

The kubeadm do the same thing it creates a CA pair of files and stores that on the master node, as of now we manually send a csr, and admin call the API manually, but as user increases then it will very time consuming, how can be automate the csr and certificate rotations when they expire?



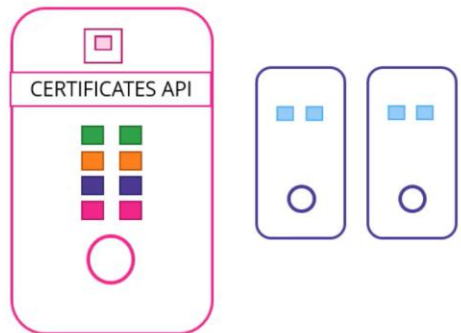
K8s has built-in certificate API, which can do this automation for you. With the certificate API, you can directly send the csr to k8s directly through an API call. This time when administrator receives a csr request, instead of login in to master node and signing the csr himself, it creates a k8s API object called CertificateSigningRequest Object. Once CertificateSigningRequest Object is created all the csr are viewed by the administrator of the cluster. Request can be easily reviewed and approved by the administrator.

1. Create CertificateSigningRequest Object

2. Review Requests

3. Approve Requests

4. Share Certs to Users



How to check the certificate expiration

kubeadm certs check-expiration

manual certificate renewal

kubeadm certs renew command.

How to view the certificate?

openssl x509 -in <.crt> -text -noout

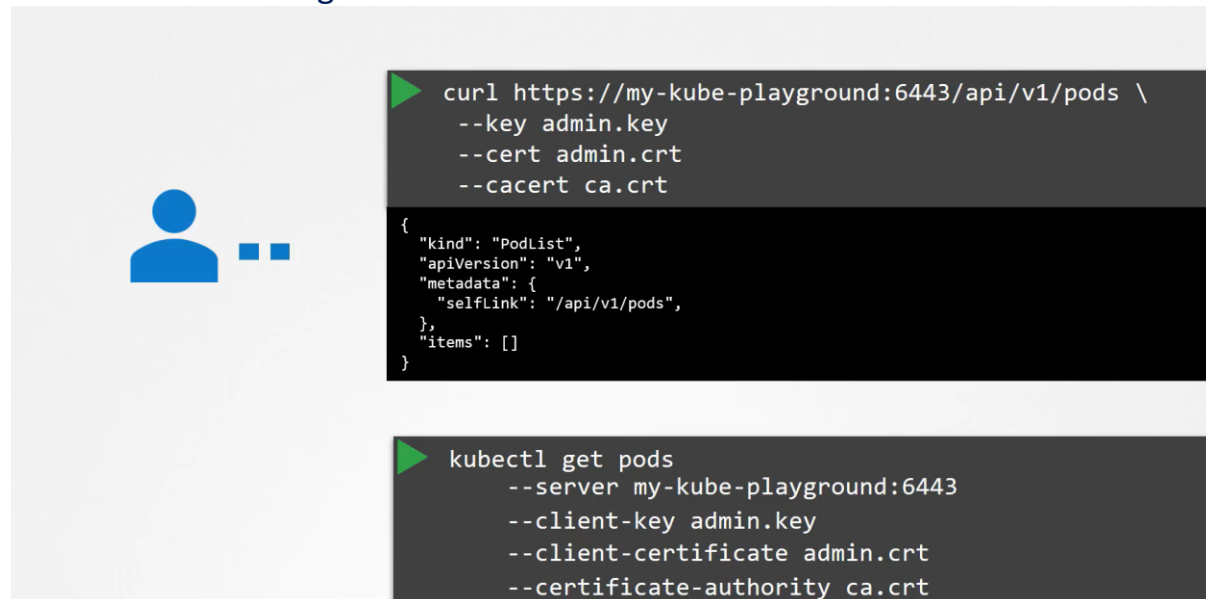
How client access the generated certificates?

As of now we saw how to generate certificate and key, now we learn how client uses this certificate files and key to query the k8s Rest API using the curl.

. kubeconfig

This is validated by the api-server for authenticate the user.

How to use this using the kubectl command?



Obviously typing this every time is tedious task, so you move this in to a config file

nano config // kubeconfig

-- server IP:6443

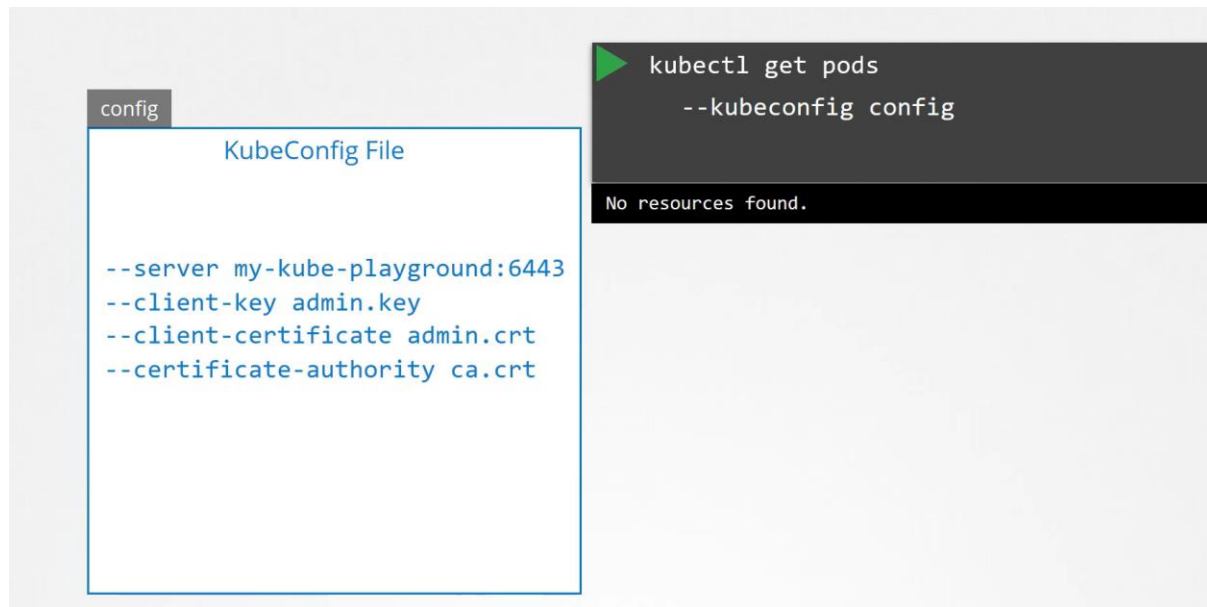
-- client-key admin.key

-- client-certificate admin.crt

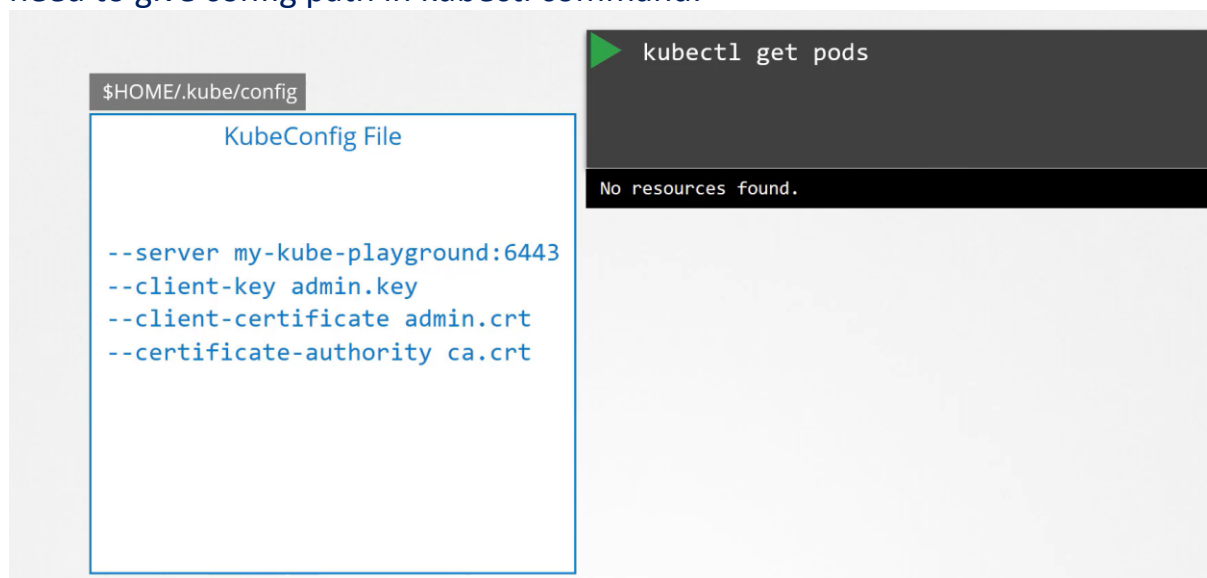
-- certificate-authority ca.crt

Then use this file as a kubeconfig option in your command

kubectl get pods --kubeconfig config

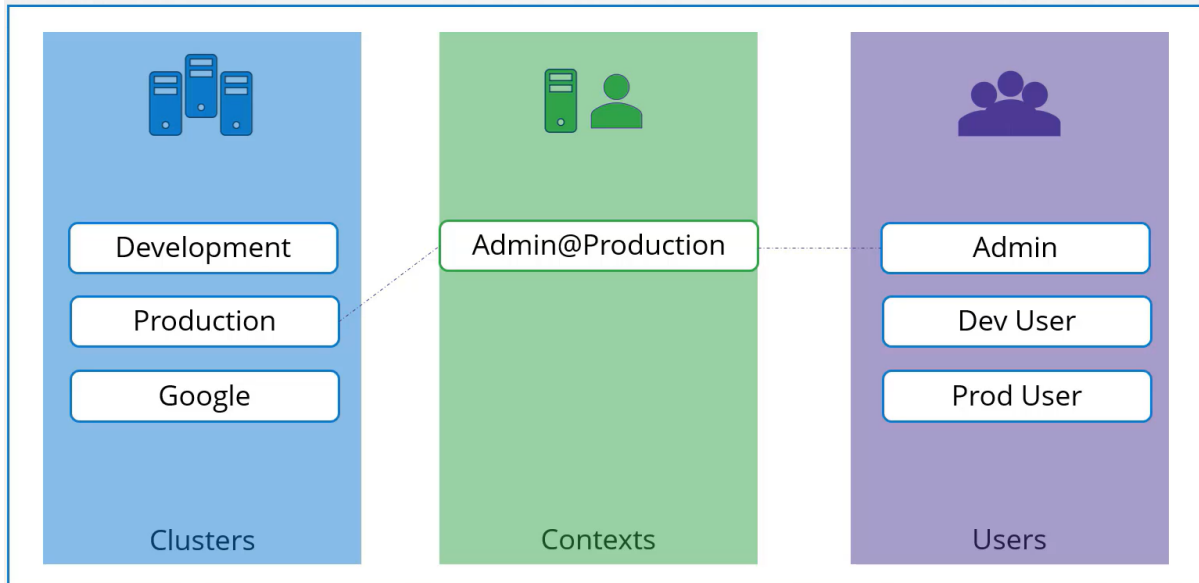


By default, kubectl find the config file under the . kube directory under the **user's home directory** if you put config file explicitly in home directory then you don't need to give config path in kubectl command.



Kubeconfig file is in specific format, config file has three sections **Cluster**, **User** and **Context**

\$HOME/.kube/config



Cluster: There may be multiple clusters for dev, prod, qa etc or for different cloud providers

-- server IP:6443 -- certificate-authority ca.crt

Users: Accounts have access to these clusters; these users may have different privileges on different clusters

-- client-key admin.key -- client-certificate admin.crt

Context: Marry these two. Define which user account use which cluster

kind: Config

apiVersion: v1

current-context: {} # current-context

clusters:

users:

contexts:

```
kind: Config
apiVersion: v1
```

```
clusters:
  - name:
    cluster:
```

```
users:
  - name:
    user:
```

```
contexts:
  - name:
    context:
```

How user use above generated certificate?

```
sudo cp /etc/kubernetes/admin.conf.
```

```
mv admin.conf prod-user.conf
```

update the prod-user.conf

```
kind: Config
apiVersion: v1
```

```
clusters:
  - name: k8s
    cluster:
      certificate-authority-data: # same
      server: https://127.0.0.1:6443 # same api server
```

```
users:
  - name: prod-user # update
    user:
      client-certificate-data / client-certificate: # -data remove exact path
      client-key-data / client-key: # -data remove exact path
```

```
contexts:
  - name: prod-user-k8s
    context:
      cluster: k8s
      user: prod-user
      namespace: # prod
```

config commands

k config -h

k config view # default config from /. kube

k config --kubeconfig=/my-custom-config view # custom config

k config get-contexts # displays list of the context

k config current-context # display the current-context

k config set-context --current --namespace=dev

permanently save the namespace for all subsequent kubectl commands in that context.

k config unset users.foo # delete user foo

k config use-context prod-k8s-playground@prod-k8s-admin

k config use-context prod-k8s-playground@prod-k8s-admin --kubeconfig=/my-custom-config

Once we move our custom-config in to. kube/ folder then it becomes our default config

validate

kubectl --kubeconfig=prod-user.conf version -short

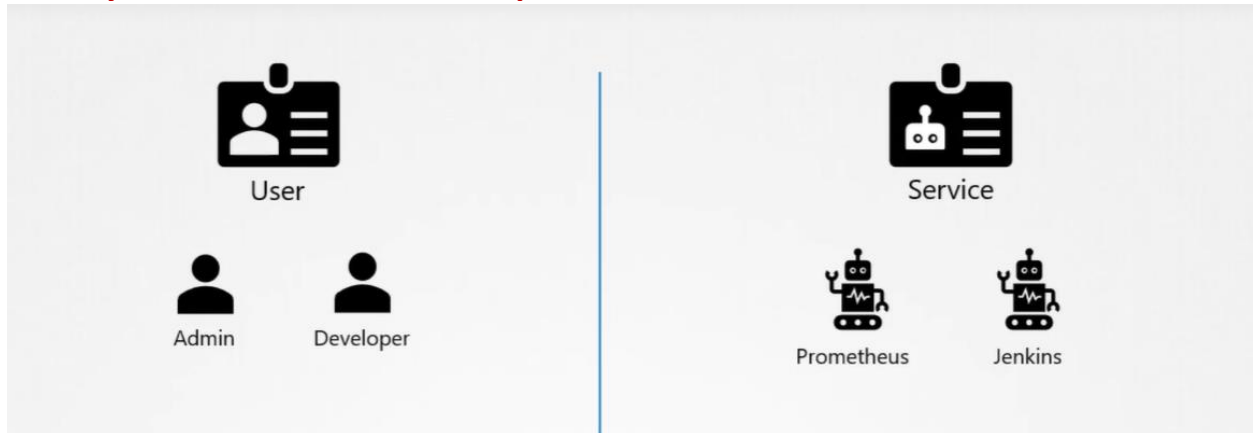
kubectl --kubeconfig=prod-user.conf get nodes

As of now user authenticated not authorized

Service Account

The concept of service accounts is linked to other security related concepts in kubernetes such as Authentication, Authorization, Role based access controls etc. A Serviceaccount is used by containers running in a Pod, to communicate with the API server of the Kubernetes cluster.

Can my Pod communicate with api-server?



So, there are two types of accounts in Kubernetes. A user account and a service account. As you might already know, the user account is used by humans and service accounts are used by machines. A user account could be for an administrator accessing the cluster to perform administrative tasks, a developer accessing the cluster to deploy applications etc. A service account, could be an account used by an application to interact with the kubernetes cluster. For example, a monitoring application like Prometheus uses a service account to poll the kubernetes API for performance metrics. An automated build tool like Jenkins uses service accounts to deploy applications on the kubernetes cluster. Just like a user but that user managed by K8s, used at application level internally not for external users, Internal to the k8s cluster.

UC: In case of monitoring, we deployed the Prometheus, grafana, they all are deployed as a Pod and communicate with api-server to collect the data.

UC: How Jenkins server communicate with api-server?

User communicate with api-server through kubectl, pod communicate with api-server through service account.

Accessing the API Server from a Pod

A lot of applications that run in the cluster (read: running in Pods), need to communicate with the API server. Among them are the processes running within the Control Plane (scheduler, controller manager, proxy, etc.), as well as all the applications that need to perform some form of administration for the cluster. For example, some applications might need to know:

- The status of the cluster's nodes.
- The namespaces available.
- The Pods running in the cluster, or in a specific namespace, and other things like that.

To communicate with the API server, a Pod uses a ServiceAccount containing an authentication token.

Roles (e.g., the right to list all the Pods within a given namespace), or ClusterRole (eg: the right to read all the Secrets within the entire cluster), can then be bound to this ServiceAccount. Respectively with a RoleBinding or a ClusterRoleBinding, so the ServiceAccount is authorized to perform those actions.

kubectl create sa dashboard-sa

When service account is created it creates a token by default, is used by external application to be authenticate by k8s API, the token is stored as a secrets object.

```
kubectl create serviceaccount dashboard-sa
serviceaccount "dashboard-sa" created

kubectl get serviceaccount
NAME          SECRETS  AGE
default       1        218d
dashboard-sa  1        4d

kubectl describe serviceaccount dashboard-sa
Name:          dashboard-sa
Namespace:     default
Labels:        <none>
Annotations:   <none>
Image pull secrets: <none>
Mountable secrets: dashboard-sa-token-kbbdm
Tokens:        dashboard-sa-token-kbbdm
Events:        <none>
```

step1: When service account is created, it first creates a service account.

step2: Then generates a token for a service account.

step3: Then it creates a secret object and stores the token inside the secret object.

step4: Then link the created secret with the service account.

```

kubect1 describe serviceaccount dashboard-sa
Name:      dashboard-sa
Namespace: default
Labels:    <none>
Annotations: <none>
Image pull secrets: <none>
Mountable secrets: dashboard-sa-token-kbbdm
Tokens:    dashboard-sa-token-kbbdm
Events:    <none>

```

```

kubect1 describe secret dashboard-sa-to

```



Secret

token:
eyJhbGciOiJSUzI1NiIsImtp
ZCI6IiJ9.eyJpc3MiOiJrdWJ
lcm5ldGVzL3NlcnZpY2VhY2N
vdW50Iiwia3ViZXJuZXRlcy5
pby9zZXJ2aWNlYWNjb3Vud...

For every ns, there is default service account is automatically created, and then it generates the token, and then generates the secret object to store the generated token, and then link the secret object with the service account. **Every Pod uses default service account, using the service token** (stored in a default secret) communicate with the api-server, token automatically mounted to the Pod as a volumeMount

```

nginx:
  Image:      my-kubernetes-dashboard
  Mounts:
    /var/run/secrets/kubernetes.io/serviceaccount from default-token-j4hkx (ro)
  Conditions:
    Type      Status
  Volumes:
    default-token-j4hkx:
      Type:      Secret (a volume populated by a Secret)
      SecretName: default-token-j4hkx
      Optional:  false

```

```

kubect1 exec -it my-kubernetes-dashboard ls /var/run/secrets/kubernetes.io/serviceaccount
ca.crt  namespace  token

```

kg secret

ki secret <name of the secret>

Authorization: Authorization ways

- RBAC (Role Based Access Control)
 - Namespace level Access (Role and RoleBinding)
 - Cluster level Access (ClusterRole and ClusterRoleBinding)

Role (What action on Which resource - namespace specific)

kc role <name of the Role> --verb=get, list, watch --resource=pod, deployment, service -n <dev>

kc role prod-ns-view-role --verb=get, list, watch --resource=pod, deployment -n prod

kg role prod-ns-view-role -n prod

ki role prod-ns-view-role -n prod

ClusterRole (What action on Which resource - cluster specific)

kc clusterrole clust-admin --verb=get, list, watch, create, delete, edit --resource=pod, deployment, persistentvolume, node

kg clusterrole clust-admin

ki clusterrole clust-admin

RoleBinding (Who can do the Role Binding user to role -> user/<namespace>:sa)

kc rolebinding <name of RoleBinding> --user=<name> --role=<name> -n <dev>

kc rolebinding <name of RoleBinding> --group=<devops> --role=<name> -n <dev>

kc rolebinding sa-rolebinding --serviceaccount=default: pvviewer --role=admin-role -n default

ClusterRoleBinding (Who can do the Role Binding user to role -> user/<namespace>:sa- cluster specific)

kc clusterrolebinding admin-binding --user=admin clusterrole=clust-admin

kc clusterrolebinding pvviewer-role-binding --serviceaccount=default: pvviewer --clusterrole=pvviewer-role

Check Access

k auth can-i create deployment

k auth can-i create Pod --as prod-user -n prod

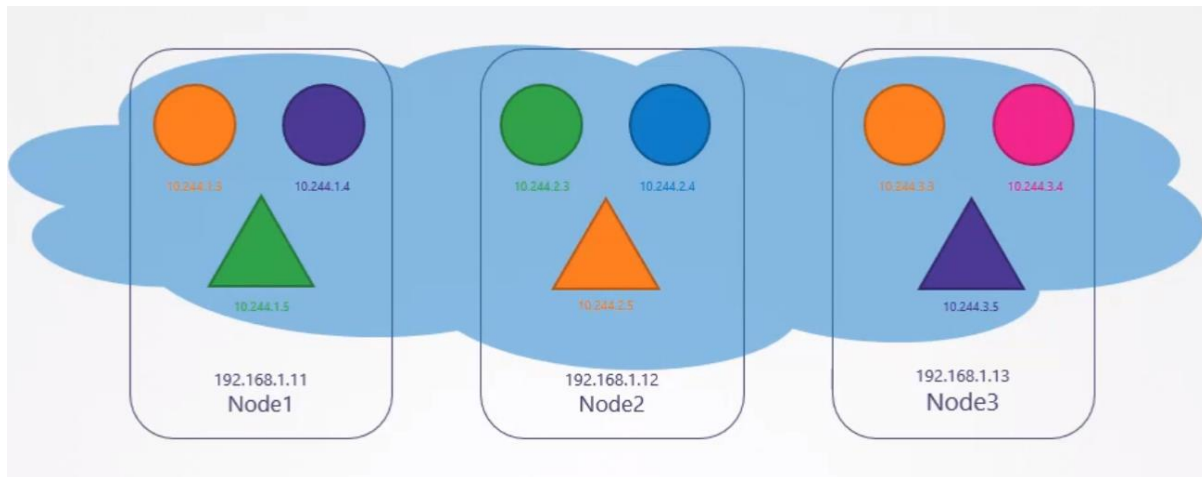
k auth can-i <verb> <resource> --as=system:

serviceaccount:<namespace>:<serviceaccountname> [-n <namespace>]

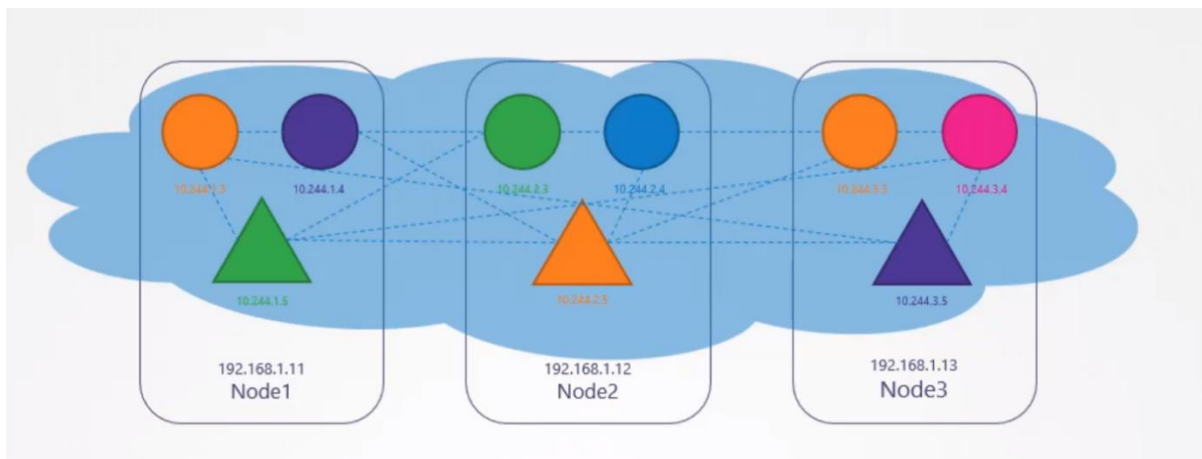
k auth can-i list pod --as=system: serviceaccount:default: admin -n default

Network Security

Let us now look at Network Security in Kubernetes. So, we have a **cluster with a set of nodes hosting a set of pods and services**. Each node has an IP address and so does each pod as well as service.



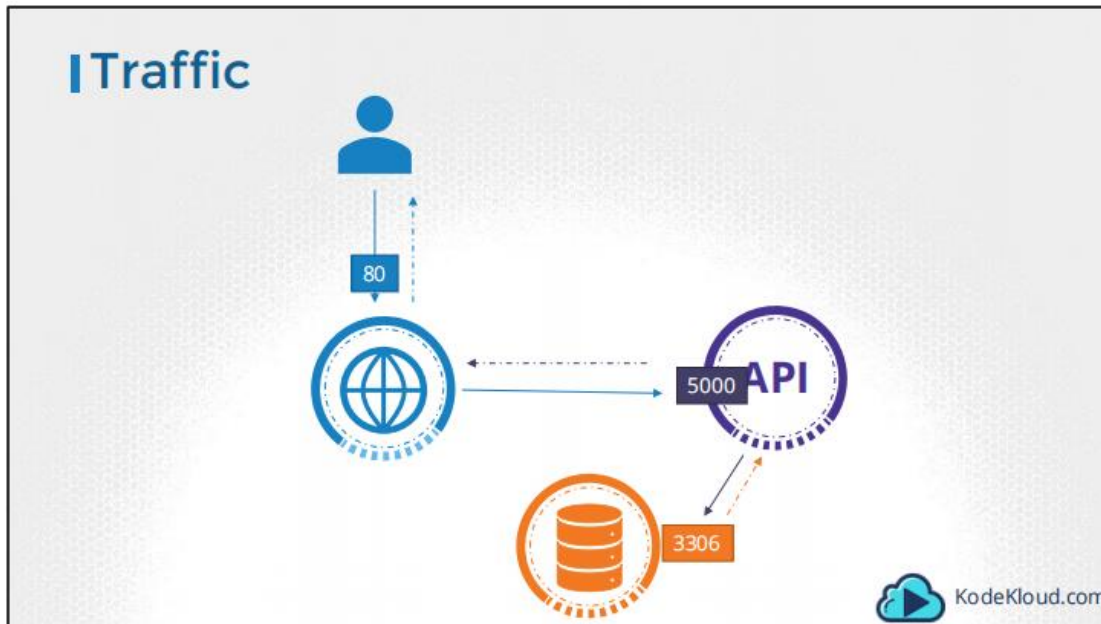
One of the pre-requisites for networking in kubernetes, is whatever solution you implement, the pods should be able to communicate with each other without having to configure any additional settings, like routes. In below network solution all pods are in virtual private network that span across the nodes in k8s cluster and by default they can reach to each-other using IP or Pod name or services configured for that purpose. K8s is by default configured by default for “ALL ALLOW” rule, any pod can communicate to any pod or services with in K8s cluster.



Let us now bring back our earlier discussion and see how it fits in to kubernetes.

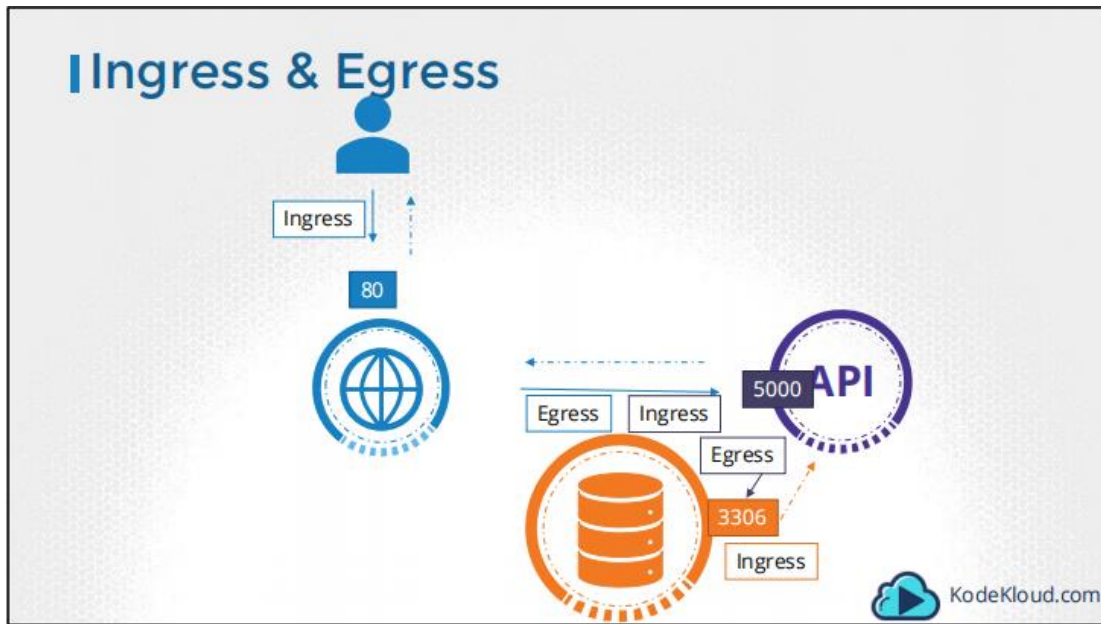
Use Case: We will start with a simple example of a traffic through a web, app and database server. So, you have a web server serving front-end to users, an app server serving backend API's and a database server. The **user sends in a request**

to the web server at port 80. The web server then sends a request to the API server at port 5000 in the backend. The API server then fetches data from the database server at port 3306 and then sends the data back to the user. A very simple setup.



So, there are **two types of traffic here. Ingress and Egress**. For example, for a web server, the **incoming traffic from the users is an Ingress Traffic** and the outgoing requests to the app server is Egress traffic and that is denoted by the straight arrow. When you define ingress and egress, remember you are only looking at the direction in which the traffic originated. The response back to the user, denoted by the dotted lines do not really matter.

Similarly, in case of the backend API server, it receives ingress traffic from the web server on port 80 and has egress traffic to port 3306 to the database server and from the database server's perspective, it receives Ingress traffic on port 3306 from the API server.



By default, any pod can communicate with any pod within the cluster.

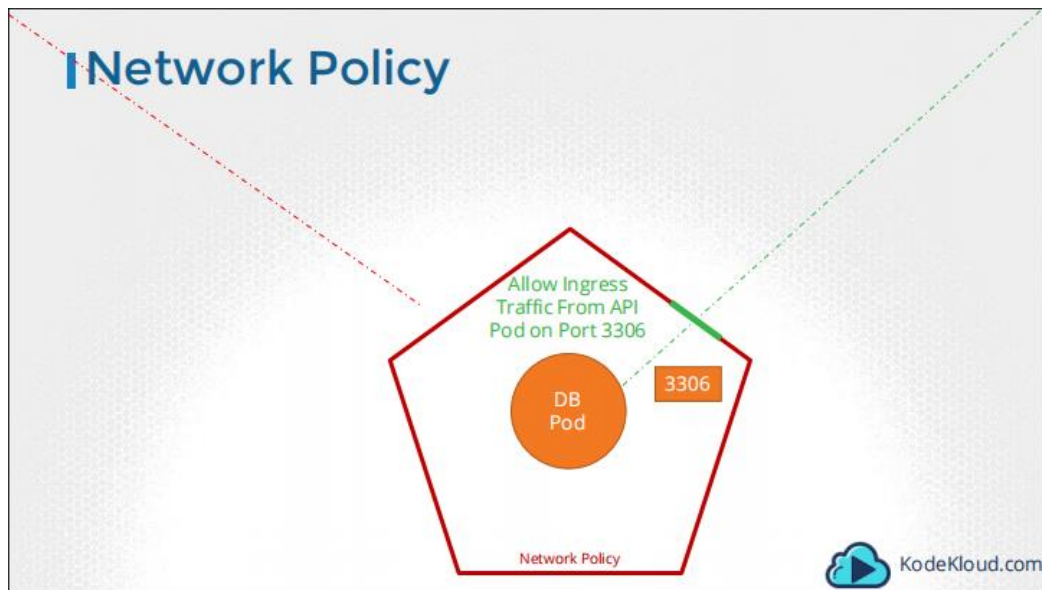
What if we do not want the front-end web server to be able to communicate with the database server directly?

Say for example, the security teams and audits require you to prevent that from happening? That is where you would implement a NetworkPolicy to allow traffic to the db server only from the API server. Let's see how we do that.

Network Policy (NACL): By default, Pod can access to each other, how can restrict?

Is something about restricting network traffic e.g., controlling network traffic?

A Networkpolicy is another object in the kubernetes namespace. Just like PODs, ReplicaSet or Services. **You link a networkpolicy to one or more pods.** You can define rules within the network policy. In this case I would say, only allow Ingress Traffic from the API Pod on Port 3306. **Once this policy is created, it blocks all other traffic to the Pod and only allows traffic that matches the specified rule,** again, this is only applicable to the Pod on which the network policy is applied.



How to apply Network policy and linked to the Pod?

Using labels and podSelector field in NetworkPolicy



PolicyType specify whether the rule defines Ingress traffic, Egress traffic or both.

Network Policy - Rules

```
policyTypes:
- Ingress
ingress:
- from:
  - podSelector:
      matchLabels:
        name: api-pod
    ports:
      - protocol: TCP
        port: 3306
```

Allow
Ingress
Traffic
From
API Pod
on
Port 3306



KodeKloud.com

Network Policy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
```

```
kubectl create -f policy-definition.yaml
matchLabels:
  role: db
```

```
policyTypes:
- Ingress
ingress:
- from:
  - podSelector:
      matchLabels:
        name: api-pod
    ports:
      - protocol: TCP
        port: 3306
```



KodeKloud.com

How do we block everything using netpol?

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

How do we allow specific traffic using netpol?


```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-netpolicy
spec:
  podSelector:
    matchLabels:
      app: db
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: web
            # ports:
            #   - port: 80

```

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: egress-netpol
spec:
  podSelector:
    matchLabels:
      app: web
  egress:
    - to:
      - podSelector:
          matchLabels:
            app: db
            # ports:
            #   - port: 5432

```

Prerequisite:

Network policies are implemented inside cluster by **Network plugins (weave, calico, flannel)**, so we must be using a networking solution which supports NetworkPolicy e.g., calico, canal provides these features, denied policy. Which pod can talk to which pod?

Network policies are implemented by Network Plugins only weave, calico and canal provide, flannel doesn't provide.

If we have both allow and deny then which will get precedence?

By default, we applied deny all, after that we applied ingress which get precedence.

All the access to the K8s cluster managed by the api-server.